

**Министерство образования, науки и молодежной политики
Краснодарского края
Государственное бюджетное профессиональное образовательное
учреждение Краснодарского края «Ейский полипрофильный колледж»**

**СОПРОВОЖДЕНИЕ И ОБСЛУЖИВАНИЕ ПРОГРАММНОГО
ОБЕСПЕЧЕНИЯ КОМПЬЮТЕРНЫХ СИСТЕМ**

ЛАБОРАТОРНАЯ РАБОТА №4

по теме:

Технологии программирования

**Выполнил:
студент ЕПК, группа
ФИО**

**Проверил:
преподаватель
специциплин
Фомин А. Т.**

Цель работы

Научиться разработке четкого, структурированного и информативного описания технологий программирования, используемых при проектировании информационных систем.

1 Теоретические сведения

«Технологии программирования»

Термином «технологии программирования» обозначают совокупность методов, инструментов, практик и концепций, которые используются для проектирования, разработки, тестирования и сопровождения программного обеспечения. Современные технологии программирования охватывают широкий спектр областей и постоянно развиваются. Рассмотрим наиболее важные направления и категории.

1. Языки программирования

Каждый язык программирования уникален своими возможностями, особенностями и назначением. Среди популярных языков выделяют:

- **Python:** Универсальный высокоуровневый язык с простым синтаксисом, широко применяемый в научных вычислениях, машинном обучении, веб-разработке и автоматизации процессов.
- **JavaScript:** Основной язык фронтенд-разработки, активно применяется для создания интерактивных веб-сайтов и современных одностраничных приложений (SPA).
- **Kotlin/Swift:** Языки для мобильной разработки на платформе Android (Kotlin) и iOS (Swift).
- **TypeScript:** Расширенная версия JavaScript с поддержкой строгих типов, часто используемая в крупных проектах.
- **C#/.NET:** Предназначен для разработки приложений под Windows, включая десктопные, корпоративные и облачные сервисы.

- **Go (Golang):** Используется для высоконагруженных сервисов и микросервисов благодаря своей скорости исполнения и поддержке параллельного программирования.

2. Парадигмы программирования

Различные парадигмы определяют подходы к написанию кода и решению задач:

- **Объектно-ориентированное программирование (ООП)** — основа большинства современных языков (Java, C#, Python). ООП основано на принципах классов, объектов, инкапсуляции, полиморфизма и наследования.
- **Функциональное программирование (ФП)** — набирает популярность благодаря своим преимуществам в параллельном программировании и обработке данных (Haskell, Scala, Clojure).
- **Императивное программирование** — классический подход, ориентированный на выполнение последовательных инструкций (C, Fortran).
- **Логическое программирование** — направлено на решение задач путем формулировки утверждений и запросов (Prolog).

3. Методы и практики разработки ПО

Современные проекты используют ряд проверенных методик и техник, направленных на повышение качества, управляемости и надежности разработки:

- **Agile-методы (Scrum, Kanban):** Предоставляют гибкий подход к управлению проектами, позволяющий быстро реагировать на изменения требований и приоритеты заказчиков.
- **Тестирование (Unit Testing, Integration Testing, End-to-end testing):** Автоматизированное тестирование повышает надежность продукта и облегчает поддержку кода.
- **Continuous Integration/Delivery (CI/CD):** Механизмы автоматического запуска сборок и деплоя, повышающие эффективность команды и снижающие риски выпуска нестабильных версий.
- **Архитектуры Microservices:** Разбиение большого монолита на небольшие автономные сервисы, облегчающие масштабирование и сопровождение проекта.

4. Средства разработки и инфраструктура

Средства разработки играют важную роль в продуктивности программиста и успешности проекта:

- **IDE (Integrated Development Environment):** Специальные среды разработки помогают писать, отлаживать и анализировать код эффективнее (Visual Studio, IntelliJ IDEA, PyCharm, VSCode).
- **Контроль версий (VCS):** Git стал фактическим стандартом для совместной работы над проектом, позволяя вести историю изменений и разрешать конфликты.
- **Системы сборки и пакетные менеджеры:** Gradle/Maven (Java), npm/yarn (JS), pip (Python), Cargo (Rust) ускоряют сборку и установку зависимостей.
- **Облачная инфраструктура:** AWS, Google Cloud Platform, Microsoft Azure предоставляют инфраструктуру для хостинга, хранения данных и вычислительных ресурсов.

5. Базы данных и хранение данных

Организация правильного хранения и эффективного доступа к данным играет решающую роль в большинстве современных проектов:

- **Реляционные базы данных (RDBMS):** PostgreSQL, MySQL, SQLite, Oracle поддерживают мощные запросы и надежные транзакции.
- **NoSQL базы данных:** MongoDB, Cassandra, Redis предназначены для работы с большими объемами данных и обеспечивают высокую производительность и масштабируемость.
- **Big Data-технологии:** Hadoop, Spark, Kafka применяются для аналитики огромных объемов данных и потоковых вычислений.

6. Веб-разработка и мобильная разработка

В современном мире большая доля проектов связана с разработкой веб-сервисов и мобильных приложений:

- **Front-end разработка:** HTML, CSS, JavaScript, современные фреймворки вроде React, Angular, Vue.js формируют основу современного фронтенда.
- **Back-end разработка:** Node.js, Django, Spring Boot, ASP.NET Core, Ruby on Rails предлагают готовые инфраструктуры для быстрого старта бэкендов.

- Мобильная разработка: Native SDK (Android/iOS), кросс-платформенные решения (React Native, Flutter) позволяют создавать нативные и гибридные приложения.

7. Машинное обучение и искусственный интеллект

Эти области стремительно развиваются и находят широкое применение в индустрии:

- Алгоритмы машинного обучения: Keras, TensorFlow, PyTorch стали популярными инструментами для тренировки моделей глубокого обучения.
- Natural Language Processing (NLP): GPT, BERT, трансформеры революционизировали обработку естественного языка.
- Computer Vision: OpenCV, YOLOv5, RetinaNet успешно решают задачи распознавания образов и компьютерного зрения.

2 Методологии разработки ПО

Методология разработки ПО — это комплекс систематизированных подходов, методов и инструментов, направленный на организацию процесса разработки программного обеспечения. Выбор правильной методологии значительно влияет на успех проекта, улучшая коммуникацию между участниками команды, сокращая сроки разработки и повышая качество конечного продукта.

Наиболее распространённые методологии разработки ПО:

1. Waterfall (Водопад)

Традиционная методология, в которой этапы разработки следуют друг за другом последовательно: Определение требований → Проектирование → Кодирование → Тестирование → Эксплуатация.

Преимущества:

- Четкое расписание этапов и сроков.
- Подходит для небольших проектов с заранее известными требованиями.

Недостатки:

- Сложно вносить изменения после завершения этапа.

- Проблемы выявляются поздно, что увеличивает стоимость исправлений.

2. Agile (гибкие методологии)

Гибкие методологии направлены на частые поставки рабочих прототипов, постоянное сотрудничество с заказчиком и способность адаптации к изменениям.

Основные представители Agile-подходов:

- **Scrum:** Команда делится на спринты длительностью 1-4 недели, после каждого спринта производится оценка выполненной работы и корректировка плана дальнейших действий.
- **Kanban:** Визуализационный подход, фокусирующийся на прозрачности и постоянном потоке задач. Используется доска Канбан для отображения статуса задач.
- **XP (Extreme Programming):** Акцент на коллективном владении кодом, парном программировании, быстром цикле обратной связи и автоматизированном тестировании.

Преимущества:

- Быстрое внедрение изменений.
- Постоянная обратная связь от заказчика.
- Повышение мотивации сотрудников.

Недостатки:

- Требуется дисциплинированности команды и высокого уровня самоорганизации.
- Сложно применить в больших командах или долгосрочных проектах.

3. DevOps

DevOps объединяет идеи разработки (Development) и эксплуатации (Operations), направлен на сокращение разрыва между командами разработки и операционного сопровождения, автоматизирует процессы интеграции, доставки и развертывания ПО.

Ключевые концепции DevOps:

- **Непрерывная интеграция (CI)** — регулярная проверка кода в репозиториях.
- **Непрерывная доставка (CD)** — быстрая подготовка релизов и их распространение.

- Автоматизация процессов — автоматизация рутинных задач, таких как тестирование и развёртывание.

Преимущества:

- Сокращение времени вывода продукта на рынок.
- Быстрая реакция на ошибки и баги.
- Минимизация человеческих ошибок за счёт автоматизации.

Недостатки:

- Необходимость значительных инвестиций в инфраструктуру и инструменты автоматизации.
- Нуждается в специальной квалификации персонала.

4. Lean Software Development (бережливое производство ПО)

Lean-методология заимствовала многие принципы из производственной сферы (Toyota Production System) и направлена на снижение потерь, улучшение качества и устранение лишнего труда.

Принципы Lean:

- Устранение отходов.
- Амплификация знаний.
- Создание культуры экспериментов.
- Уважительное отношение к людям.

Преимущества:

- Максимально эффективное использование ресурсов.
- Снижение рисков.
- Увеличение удовлетворенности клиентов.

Недостатки:

- Трудно внедрить, так как требует серьёзных организационных изменений.
- Ориентация на краткосрочную перспективу может привести к проблемам в будущем.

5. Spiral Model (спиральная модель)

Модель сочетает черты инкрементальной разработки и водопадной схемы, предусматривая прохождение циклов развития, каждый из которых сопровождается постепенным увеличением функциональности и степенью риска.

Преимущества:

- Понимание возможных рисков и угроз на ранних этапах.
- Гибкость внесения изменений.

Недостатки:

- Дороговизна и сложность внедрения.
- Отсутствие четких временных рамок завершения проекта.

Критерии выбора методологии

- Размер и опыт команды.
- Степень определённости требований.
- Бюджет и временные рамки проекта.
- Уровень вовлеченности заказчика.
- Желаемая частота поставок.

Каждая методология имеет свои плюсы и минусы, и правильный выбор определяется спецификой проекта и командой разработчиков. Часто эффективные методики комбинируются для достижения оптимального результата.

3 Типы анализа эффективности ПО

1. Профилировка производительности

Профилировка позволяет выявить участки кода, которые занимают больше всего времени на исполнение или расходуют большое количество памяти. Популярные профилировщики:

- Valgrind/KCacheGrind: Бесплатные инструменты для Linux, позволяющие обнаружить утечки памяти и проблемы с производительностью.
- Intel VTune Amplifier: Мощный коммерческий профилировщик, предназначенный для диагностики CPU/GPU-нагрузок и производительности.
- YourKit/JProfiler: Платформы для мониторинга производительности Java-приложений, позволяющие исследовать потоки выполнения, загрузку ЦПУ и потребление памяти.
- PySpy: Инструмент для трассировки потоков выполнения Python-программ, удобный для анализа ресурсоемких участков.

2. Мониторинг нагрузки и стресс-тестирование

Мониторинг нагрузки имитирует реальную эксплуатацию ПО и выявляет пределы его производительности и устойчивости. Наиболее известные инструменты:

Apache JMeter: Открытый инструмент для тестирования нагрузок и производительности веб-приложений, поддерживает HTTP(S), SOAP, FTP и другие протоколы.

- **LoadRunner (Micro Focus):** Один из старейших коммерческих продуктов для нагрузочного тестирования, хорошо интегрируется с различными технологиями.
- **Locust:** Свободный инструмент для симулирования одновременных подключений к сервису и проверки отказоустойчивости приложений.
- **Artillery:** Современный open-source инструмент для нагрузочных испытаний REST/GraphQL API и WebSocket соединений.

3. Автоматизированное тестирование

Автоматизированные тесты повышают уверенность в качестве ПО и снижают риск появления дефектов при изменениях. Они включают разные виды проверок:

- **Unit tests:** Проверяют отдельные модули и классы (JUnit, pytest, Nunit).
- **Integration tests:** Оценивают совместную работу разных компонентов (Selenium, Cypress).
- **End-to-end tests:** Полноценно воспроизводят сценарии пользователя (Playwright, TestCafe).
- **API tests:** Тестируют стабильность API и его функциональные возможности (Postman, Insomnia).

4. Логирование и мониторинг активности

Сбор и анализ логов помогают обнаруживать аномалии, оценивать эффективность выполнения операций и получать статистику работы приложения:

- **ELK stack (Elasticsearch, Logstash, Kibana):** Решение для централизованного сбора и анализа логов, удобной визуализации данных и обнаружения отклонений.

- **Graylog:** Альтернатива ELK, удобная для компаний среднего бизнеса, работающая в режиме реального времени.
- **Sentry:** Сервис для автоматической регистрации исключений и уведомлений об ошибках, помогающий оперативно устранять сбои.

5. Аналитика производительности базы данных

Работа с большими объёмами данных требует специализированных инструментов для анализа производительности СУБД:

- **Explain Plan:** Встроенный механизм многих реляционных баз данных (MySQL, PostgreSQL, Oracle), позволяющий увидеть планы выполнения запросов и оптимизировать их.
- **pgAdmin:** Графический инструмент для администрирования PostgreSQL, включая мониторинг выполнения запросов и индексов.
- **New Relic APM:** Комплексный монитор производительности приложений, включая детализированную диагностику работы базы данных.

6. Сбор статистики и телеметрии

Использование телеметрии даёт глубокое понимание того, как реально используется приложение пользователями:

- **Google Analytics:** Основная система аналитики для веб-проектов, собирает подробную информацию о поведении пользователей и взаимодействии с сайтом.
- **Firebase:** Система аналитики и управления проектами от Google, объединяющая мониторинг поведения пользователей, управление пуш-уведомлениями и проверку эффективности маркетинговых кампаний.
- **Splunk:** Решения для анализа больших объемов журналов, данных IoT и других корпоративных систем.

7. Инструменты рефакторинга и анализа покрытия

Анализ покрытия кода позволяет понять, насколько полно покрыты существующие тесты:

- **JaCoCo:** Инструмент для измерения покрытия кода в Java-проектах.
- **Coverage.py:** Аналог JaCoCo для Python, который позволяет определить незакрытые участки кода.

- **SonarQube:** Мощная платформа для анализа качества кода, включающая подсчет покрытия, выявление дублированного кода и выявление потенциальных уязвимостей.

Применение указанных средств анализа и мониторинга позволяет своевременно диагностировать потенциальные проблемы в программном обеспечении, улучшать производительность и повышать надёжность разрабатываемых систем.

4 Примерное описание

Проектирование программного продукта потребовало тщательного подбора технологий и методов программирования, позволяющих достичь поставленных целей. Ниже представлены используемые технологии и инструменты, их особенности и обоснование выбора.

Методология разработки

Целью настоящего дипломного проекта является разработка программного продукта, соответствующего современным стандартам качества и пригодного для эффективной эксплуатации в реальной среде. Исходя из специфики проекта, определяющими критериями выбора методологии являлись:

- Ясность и предсказуемость процессов разработки.
- Способность оперативно реагировать на изменения требований заказчика.
- Возможность поддержания постоянного контакта с клиентом и быстрой коррекции приоритетов задач.
- Потребность в качественном контроле качества и документировании промежуточных результатов.

Исходя из вышеперечисленных критериев, рассматривались следующие варианты методологий:

- Водопадная модель (*Waterfall*)
- Гибкие методологии (*Agile*) — Scrum, Kanban
- Бережливая разработка (*Lean Software Development*)
- Модель *Spiral*

Проведённый сравнительный анализ показал, что для успешной реализации данного проекта наиболее целесообразно применение гибких методологий, а именно *Scrum*.

Причины выбора заключаются в следующем:

- Быстрая обратная связь. *Scrum* предполагает короткие циклы (спринты), длительность которых составляет около 2-х недель. Такая периодичность позволяет команде регулярно демонстрировать заказчику рабочий продукт и оперативно учитывать его пожелания.
- Высокий уровень прозрачности. Вся деятельность в рамках *Scrum* ведется открыто и прозрачно: ежедневные встречи, стендап-митинги, публичные доски задач способствуют быстрому обмену информацией и предотвращению неопределенностей.
- Привлечение заказчика. Благодаря активным встречам с представителями заказчика и демонстрации текущих достижений удастся снизить вероятность расхождений между ожиданиями и результатом.
- Повышенная мотивация команды. Участники команды получают самостоятельность в принятии решений, что улучшает общий моральный климат и мотивацию участников.
- Простота освоения. Хотя переход на *Scrum* требует первоначальных усилий, эта методология проста в освоении и применении, что существенно снижает затраты на внедрение.

Однако существуют и ограничения *Scrum*, которые необходимо учитывать:

- Данная методология требует дисциплины и высокого уровня самоорганизации членов команды.
- Для успешного функционирования необходимы опытные *scrum*-мастера и специалисты по сопровождению проекта.

Тем не менее, учитывая перечисленные достоинства и потребности проекта, *Scrum* представляется оптимальным решением.

План мероприятий по применению выбранной методологии

План реализации *Scrum* включает следующие мероприятия:

- Формирование проектной команды, определение ролей (Product Owner, Scrum Master, разработчики).
- Составление backlog'а продукта, постановка начальных приоритетов задач.

- Регулярное проведение планирования спринтов, демо-презентаций и ретроспектив.
- Установка соответствующих инструментов для ведения задач (например, Jira, Trello).
- Назначение регулярного технического аудита и рефакторинга.

Рассмотрев возможные альтернативы, принято решение использовать методологию *Scrum*, как наиболее подходящую для конкретных условий проекта. Эта методика обеспечит необходимую гибкость, повысит качество разработки и облегчит коммуникацию с заказчиками.

Языки программирования

Основной язык программирования, выбранный для реализации проекта, — Python (или JavaScript, Kotlin, Swift и т.п.). Выбор обусловлен несколькими факторами:

- Широкая поддержка библиотек и сообществом разработчиков, упрощающих разработку сложных проектов.
- Высокий уровень абстракции и читаемость кода позволяют сосредоточиться на бизнес-логике и уменьшить вероятность ошибок.
- Эффективность разработки благодаря обширной экосистеме готовых модулей и пакетов.

Дополнительно были использованы вспомогательные языки, такие как SQL для работы с базой данных и HTML/CSS/JavaScript для веб-интерфейса (если речь идет о веб-приложении).

Архитектурные подходы

Реализованная архитектура соответствует принципу разделения ответственности (Separation of Concerns) и паттерну MVC (Model-View-Controller)/MVVM (Model-View-ViewModel).

Такой подход позволил добиться:

- Чёткого разделения логики приложения на слои: модели данных, представление и контроллеры.
- Упрощённого тестирования отдельных компонентов.
- Легкости поддержки и расширения функционала.

Инструменты и среды разработки

Процесс разработки проходил с использованием интегрированной среды разработки PyCharm (или Visual Studio Code, Android Studio и т.п.) — популярного IDE, обеспечивающего удобные средства написания, отладки и анализа кода.

Средства автоматизации сборки и развертывания включены в CI/CD процессы (GitHub Actions, Jenkins и т.п.), обеспечивая непрерывную интеграцию и доставку продукта.

Библиотеки и фреймворки

Для ускорения разработки и повышения качества кода использовались следующие сторонние библиотеки и фреймворки:

- Django/Falcon (для серверной части веб-приложения).
- Flask/Django REST Framework (для API).
- React/Vue.js (для клиентской части веб-приложения).
- PostgreSQL/Oracle Database (для хранения данных).
- Selenium/Pytest (для автотестирования).

Выбор перечисленных инструментов обоснован следующими причинами:

- Высокая популярность и зрелость используемых библиотек, наличие документации и примеров использования.
- Соответствие поставленным требованиям и техническому заданию.
- Наличие активного сообщества и доступность помощи при возникновении проблем.

Тестирование и отладка

На этапе разработки особое внимание уделялось тестированию, в частности:

- Написаны юнит-тесты с покрытием критически важных функций.
- Проведены нагрузочные тесты для оценки производительности.
- Применялись автоматические регрессионные тесты для предотвращения появления багов.

В этом разделе обязательно приводятся:

- *Описания контрольных примеров с демонстрацией результатов для юнит-тестов и нагрузочных тестов.*

- **Используемых элементов безопасного программирования и отладки (вывод отладочных сообщений, логирования ошибок, использование assert-функций)**

Примечание. Поскольку данный текст находится в свободном доступе в сети Интернет, буквальное его заимствование с размещением в дипломной работе — безусловно запрещено!! Текст «Примерного описания» (равно как и текст теоретической части) приведен только как образец для подражания, в ознакомительных целях, не более того!

4 Постановка задачи

- **Составить подробное, но не избыточное, четко сформулированное и лаконичное описание выбранных технологий программирования, применяемых в процессе проектирования, разработки и внедрения ПО дипломного/курсового проекта**
- **Показать адекватный выбор решений для поставленной задачи**

5 Содержание отчета

1. **Цель работы;**
2. **Постановка задачи;**
3. **Описание технологий программирования;**
4. **Выводы по выполненной работе;**
5. **Ссылки на использованные источники (по ГОСТ).**